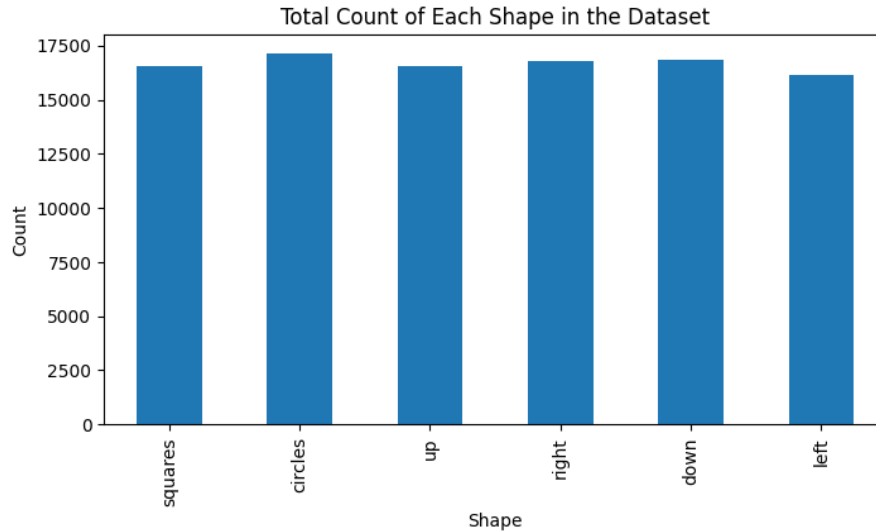


Exploratory Data Analysis

The dataset contains no null values and is balanced with respect to shape occurrences and class distribution. However, it does not include any of the $(1, 9) / (9, 1)$ configurations for each shape pair, leaving $\binom{6}{2} \cdot 2 = 30$ classes missing.



Model architecture

I used similar architectures for both the classification and regression heads. Each consists of two fully connected layers with a ReLU activation and dropout in between. I added dropout to reduce overfitting. In multitask model I used $\lambda_{\text{cnt}} = \frac{\text{model_cls_loss}}{\text{model_cnt_loss}}$ to balance the influence of both targets.

Augmentations

The most important augmentations were flips and rotations, as they increased the diversity and size of the dataset. These transformations required updating the labels to reflect the new orientations. For example, when rotating the image 90° clockwise, an upward-facing triangle becomes right-facing, a left-facing triangle becomes upward-facing, and so on. Similarly, when flipping horizontally, a left-facing triangle becomes right-facing, and vice versa.

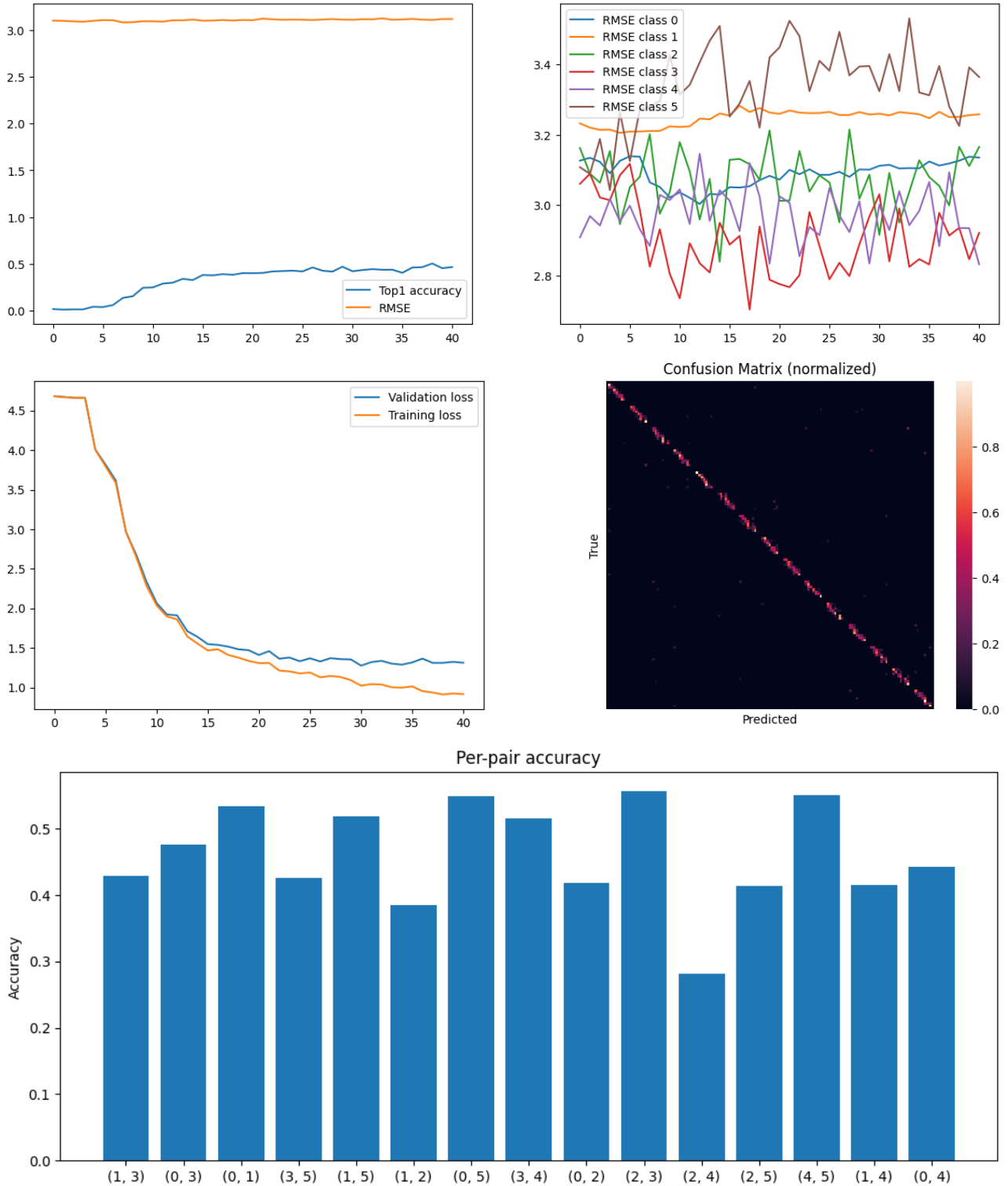
Additionally, I applied random brightness and contrast to slightly vary the image intensity. Since the images are binary, this introduces subtle gray-level variations without changing the shape counts or orientations. While both the training and test images remain binary, this augmentation provides mild regularization during training.

Result tables

Each model ran for approximately 5 minutes.

Classification model

This model's regression head doesn't learn anything, because $\lambda_{\text{cnt}} = 0$.

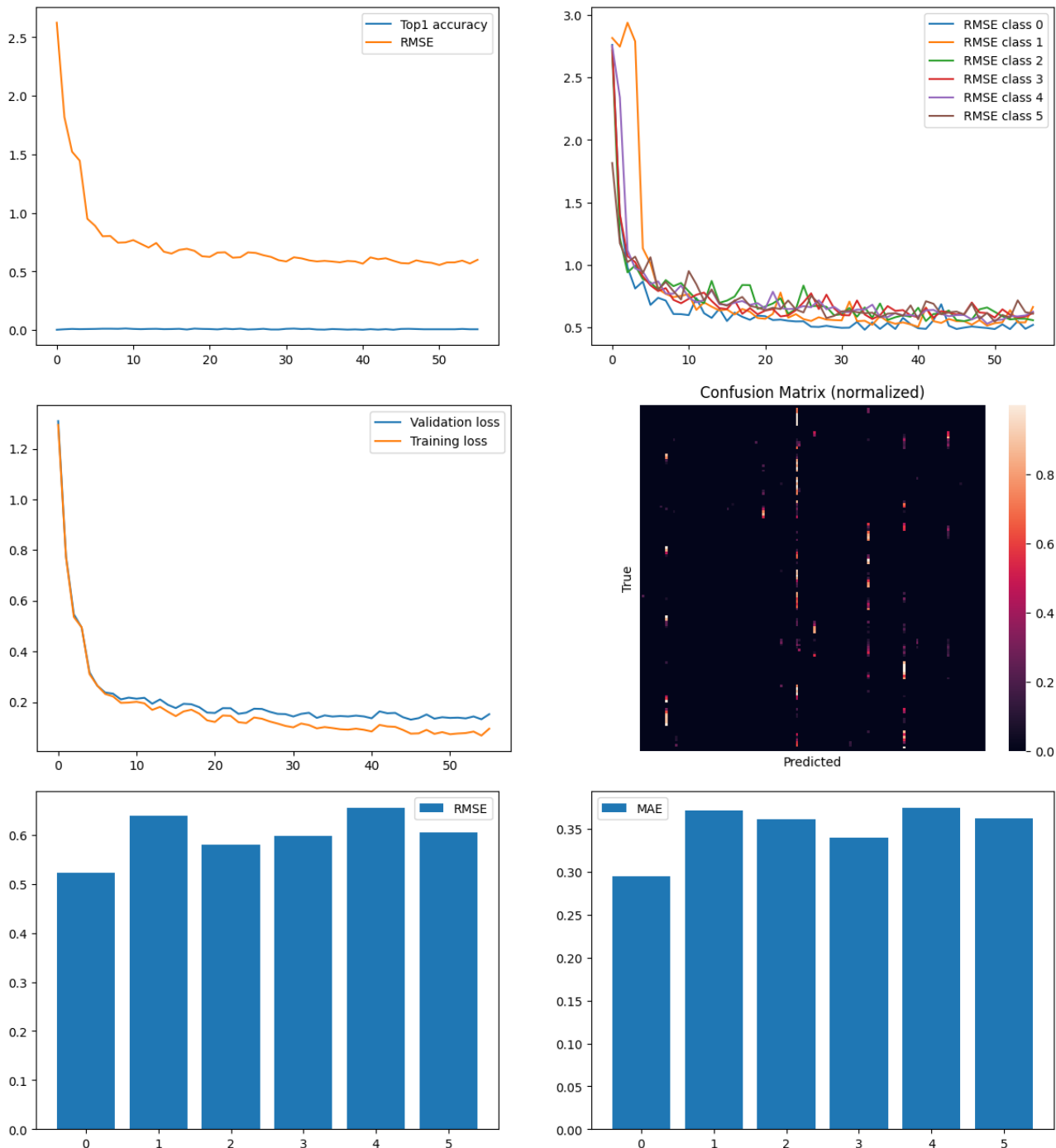


The training and validation loss curves show a rapid decrease during the first 15 epochs, followed by a smooth and stable convergence. The gap between training and validation loss remains small, indicating that the model does not overfit significantly and generalizes well to the validation set. The training process appears stable, with no oscillations or divergence. The validation set is too small to say anything meaningful about per-pair accuracy.

In the confusion matrix, most mistakes are near the diagonal, showing that errors are usually in counting, not in identifying the shape types.

Regression model

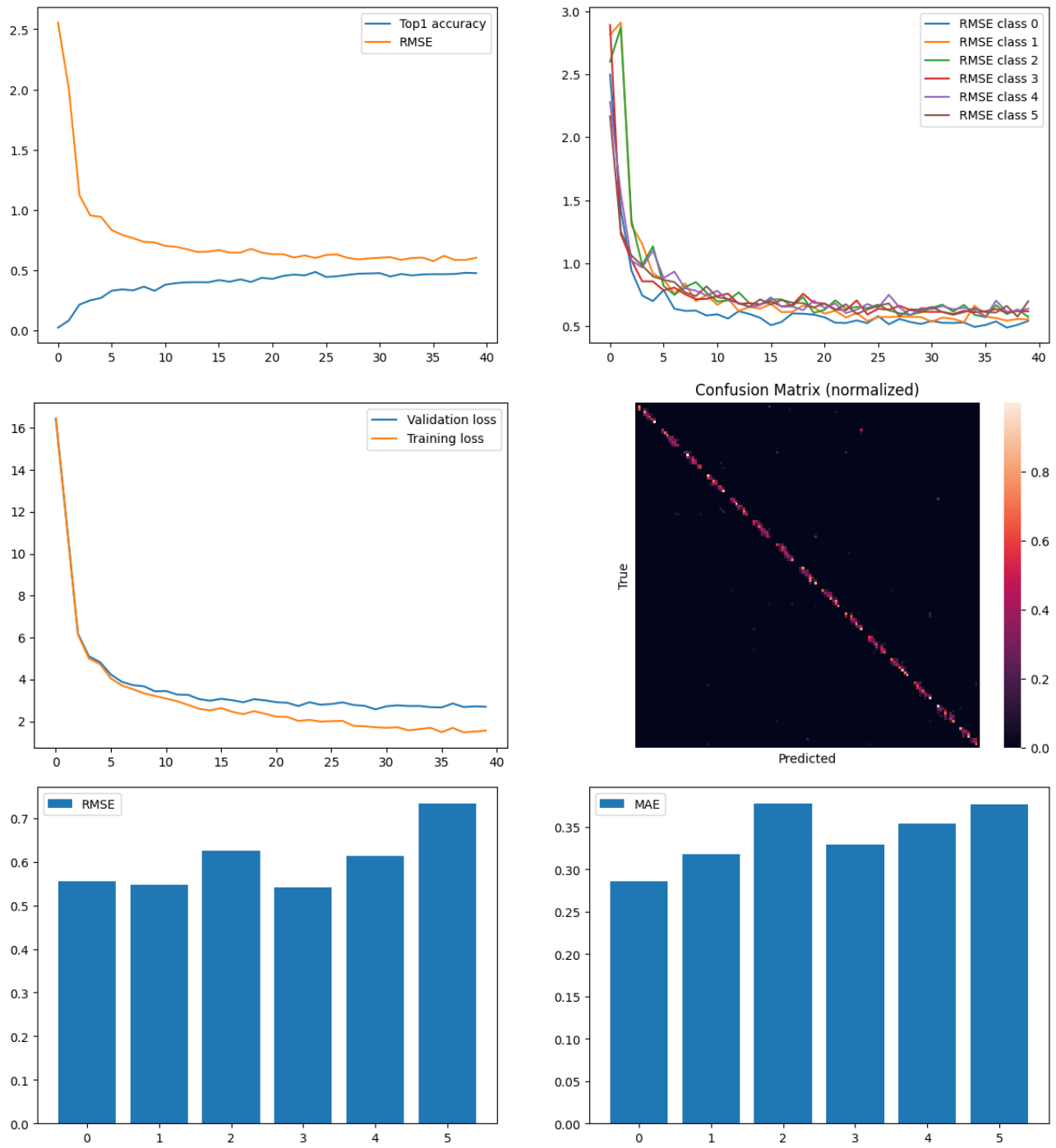
This model's classification head doesn't learn anything, because $\lambda_{\text{cls}} = 0$.

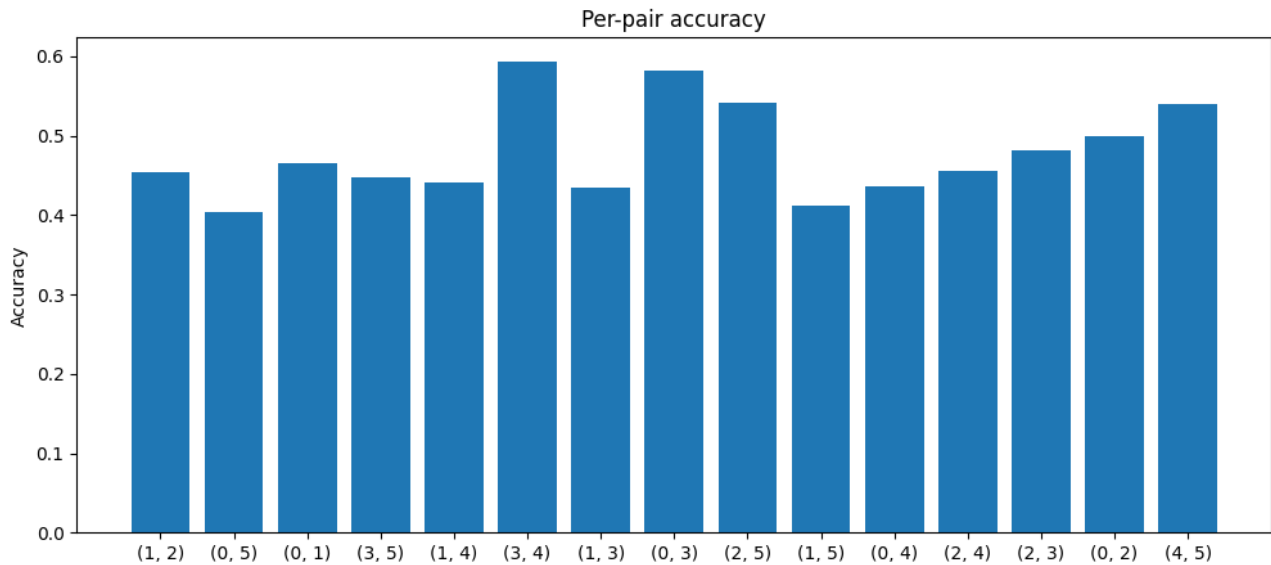


The training and validation loss curves show a rapid decrease during the first 5 epochs, followed by a smooth and stable convergence. The gap between training and validation loss remains small, indicating that the model does not overfit significantly and generalizes well to the validation set. The model appears to predict the count of class 0 (circles) more accurately than for the other shapes, indicating that this particular shape is easier for the network to recognize and count. Classification head doesn't learn and it usually guesses the same couple

classes.

Multitask model





The training and validation loss curves show a rapid decrease during the first 5 epochs, followed by a smooth and stable convergence. The gap between training and validation loss remains small, indicating that the model does not overfit significantly and generalizes well to the validation set. The validation set is too small to say anything meaningful about per-pair accuracy. Some pairs just appear more often than others, so their scores look better by chance. In the confusion matrix, most mistakes are near the diagonal, showing that errors are usually in counting, not in identifying the shape types.

Discussion on multitask effects

Table 1: Comparison of results

Metric	Classification-only	Regression-only	Multitask
Top-1 Accuracy	0.46	0.01	0.52
Macro F1	0.44	0.00	0.51
RMSE (overall)	3.16	0.56	0.55
MAE (overall)	1.89	0.31	0.31

As we can see, the multitask model performs better on both tasks than the models trained for only one task. The multitask model performs better than single-task models because it shares knowledge across related tasks. The backbone learns features useful for both classification and regression, giving richer and more general representations than learning each task separately.